

Test Cases _ Graph Phone Book (Q-Test)

The cases follow the product brief in order: authentication, contacts, search, then protocol behavior.
(AUTH: authentication, PB: phonebook/contacts, SRCH:search, ZMQ: protocol, Neg: negative/edge)

1. Authentication

These cases check how user register, sign in, and leave a session, and whether the phone book actually requires a signed-in user.

TC-AUTH-01 _ Sign up

Start with a fresh database. Call `sign_up` with a unique username, password, and email. The system should return a success message and create a user row.

TC-AUTH-02 _ Sign up with a duplicate username

Register a user first, then try `sign_up` again with the same username. This should fail clearly rather than silently overwriting the existing account. In the current code, this surfaces as a database integrity error.

TC-AUTH-03 _ Sign up with a duplicate email

Register one user, then try `sign_up` with a different username but the same email. The brief implies email should be unique, but the schema has no unique constraint on email, so this is a product decision to confirm by hand. *Manual*.

TC-AUTH-04 _ Sign in with valid credentials

With a registered user, call `sign_in` with the correct username and password. Expect a success response, and the username should appear in `online_users`.

TC-AUTH-05 _ Sign in with the wrong password

With a registered user, call `sign_in` with a bad password. Expect an error that mentions the wrong password, and the user should not be marked online.

TC-AUTH-06 _ Sign in for an unknown user

On a fresh database, call `sign_in` for a username that does not exist. Expect a clear error. The current code raises a tuple-shaped message rather than a clean string.

TC-AUTH-07 _ Logout for a signed-in user

Sign a user in so they appear in `online_users`, then call `logout`. Expect success and removal from `online_users`.

TC-AUTH-08 _ Logout when never signed in

Call `logout` for a user who is not online. Ideally this fails or does nothing harmful. *Actual behavior*: it still returns “signed out successfully.”

TC-AUTH-09 _ Use contacts without a session

Without signing in first, call `add_phone_user`. The brief says the phone book should require authentication, so this ought to be rejected. *Actual behavior*: the contact is created anyway.

Automated: TC-AUTH-01, 02, 04–09 in `test_auth_commands.py`

Manual: TC-AUTH-03.

2. Contacts (CRUD)

These cases cover adding, editing, and removing contacts, including duplicate names and phone numbers, and what happens when the database is not ready.

TC-PB-01 _ Add a contact with an explanation

With the database ready, call `add_phone_user` with a name, phone number, and explanation. The contact should be stored, and a lookup by name should return the number or numbers.

TC-PB-02 _ Add a contact without an explanation

Call `add_phone_user` while omitting explanation. The brief lists explanation as a parameter; the code defaults it to an empty string, and the sample file also omits it. Confirm that this still works.

TC-PB-03 _ Duplicate contact username

Create a contact, then call `add_phone_user` again with the same username and a new number. This should be rejected because username is unique on `phone_user_table`.

TC-PB-04 _ Add a second phone number

For an existing contact, call `add_phone_number`. A later lookup should show both numbers.

TC-PB-05 _ Edit: rename and change the number

Start with a contact that has one number. Call `edit_phone_user` with both `new-name` and `newnumber` fields set. Looking up the new name should show the new number.

TC-PB-06 _ Edit: name only

For an existing contact, set only `new_username`. The name should change and the phone numbers should stay the same.

TC-PB-07 _ Remove a contact

For an existing contact, call `remove_phone_user`. The contact should be gone, and a lookup by name should fail.

TC-PB-08 _ Remove a missing contact

Call `remove_phone_user` with an unknown name. Expect a graceful message and no crash.

TC-PB-09 _ Same phone number on two contacts

Give contact A a number, then try to add contact B with that same number. This should be rejected because phone numbers are globally unique.

TC-PB-10 _ Database not initialized

In an empty process, send `add_phone_user` as the first command without going through auth. Tables should already exist, or the error should be clean. *Actual behavior*: no such table, because only `sign_up` calls `before_request()`.

Automated: TC-PB-01, 02, 04, 05, 07, 09.

Not covered by pytest yet: TC-PB-03, 06, 08, 10 (10 confirmed manually and by code review).

3. Search and list

These cases check listing everyone and finding people by name or number, plus whether different lookup paths return consistent fields.

TC-SRCH-01 _ List all contacts

Set up at least two contacts, one of them with two numbers. Call `get_all_phone_users`. Results should be grouped by name and include numbers and explanation.

TC-SRCH-02 _ Look up by name (found)

For an existing contact, call `get_phone_user_by_name`. Expect the name and the list of phone numbers.

TC-SRCH-03 _ Look up by name (missing)

Call `get_phone_user_by_name` with an unknown username. Expect a clear failure.

TC-SRCH-04 _ Look up by number (found)

For a number that exists, call `get_phone_user_by_number`. Expect the owner's name and their numbers.

TC-SRCH-05 _ Look up by number (missing)

Call `get_phone_user_by_number` with an unknown number. Expect a clear failure.

TC-SRCH-06 _ Field consistency across lookups

Create a contact with an explanation, then compare `get_all_phone_users` with `get_phone_user_by_name`. The keys for phones and explanation should match. *Actual behavior*: one path uses `phone_number` and the other `phone_numbers`, and the by-name path drops the explanation. *Manual*.

Automated: TC-SRCH-01, 03, 04 (TC-SRCH-02 is inside add/lookup tests).

Manual: TC-SRCH-06 and TC-SRCH-05.

4. Protocol / ZeroMQ

These cases check that the client and server talk correctly over ZeroMQ, and how the system behaves on bad input.

TC-ZMQ-01 _ Client and server connectivity

With the server up, send a small command batch. A response should arrive.

TC-ZMQ-02 _ Full sample file end to end

With the server up, run the client against `samples/commands.json`. The happy path batch should work end to end. *Manual* _ inspect the printed response shape.

TC-ZMQ-03 _ Unknown command

With the server ready, send `command_name: "not_a_command"`. Expect a structured error, and the server should stay up. *Actual behavior:* an error string goes on the wire, then the server may die after a raise.

TC-ZMQ-04 _ Bad or missing --file

Point the client at a missing file or invalid JSON. It should fail early instead of hanging on ZeroMQ. *Manual.*

TC-ZMQ-05 _ Response encoding

With the server ready, send any valid batch. `recv_json()` should yield a list. *Actual behavior:* the payload is double encoded as a string because of `send_json(json.dumps(...))`.

Automated: TC-ZMQ-01 / 05 (including a short multi-command batch) and TC-ZMQ-03 in `test_zmq_protocol.py`.

Note: the happy path test asserts the double encoding so the defect stays visible until it is fixed.

5. Negative and edge cases (mostly manual)

These are harder paths that are easy to miss in happy path automation. Most of them are best checked by hand.

TC-NEG-01 _ Empty username on sign-up

Try `sign_up` with an empty username. This is a typo or abuse path; expect a validation error. The brief does not spell out the exact rule.

TC-NEG-02 _ SQL injection in username

Put attacker-controlled text into username fields. The system should not crash or wipe tables.

TC-NEG-03 _ Very long explanation

Send an extremely long explanation string. Accepting it, truncating it, or returning a clear error are all plausible; the brief leaves this undefined.

TC-NEG-04 _ Unicode names (Persian / emoji)

Use locale-relevant names, including Persian text and emoji. Round-trip storage and lookup should remain readable.

TC-NEG-05 _ Concurrent sign-in for the same user

Sign the same user in more than once. Session behavior for `online_users` should be defined and consistent.

TC-NEG-06 _ Port out of range

Start the service with a port outside the allowed range. It should reject the value or fall back to 9090 (valid ports are range(1000, 9998)).

TC-NEG-07 _ Mutate contacts after logout

Sign in, log out, then try to change contacts. The auth gate should deny this today it shares the same gap as TC-AUTH-09.

Traceability to pytest

Here is how the numbered cases map onto the automated suite:

- **Auth** _ TC-AUTH-01, 02, 04–08 live in `test_auth_commands.py`.
- **Auth gap** _ TC-AUTH-09 is covered by `test_phone_book_operations_do_not_require_auth`.
- **Contacts** _ TC-PB-01, 02, 04, 05, 07, 09 live in `test_phone_book_commands.py`.
- **Search** _ TC-SRCH-01, 03, 04 live in `test_phone_book_commands.py`.
- **ZeroMQ** _ TC-ZMQ-01, 03, 05 live in `test_zmq_protocol.py`.

A few notes for anyone reading the suite: command tests go through `PhoneBookClient` into `call_func`, which is the same dispatcher the server uses. Each test gets a fresh `sab.db` and a cleared `online_users` list via `conftest.py`. ZeroMQ tests spawn `server.py` and bind a random REQ port the client binds and the server connects, matching the system under test.